SKILL CHAIN: CAMPUS PEER-TO-PEER SKILL EXCHANGE PLATFORM

Integrated OOPS and DBMS Project Report

Submitted by

VARSHA K – 250701807

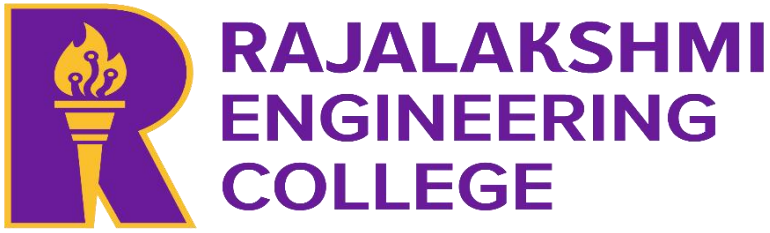
VANDHANA M – 250701804

VARSHINI S – 250701809

VARSSINI A – 250701811

of
BACHELOR OF ENGINEERING
in
Department of Computer Science and Engineering

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM
(An Autonomous Institution)



November 2026

RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project report titled “SKILL CHAIN: CAMPUS PEER-TO-PEER SKILL EXCHANGE PLATFORM” is the bonafide work of “VARSHA K (250701807), VANDHANA M (250701804), VARSHINI S (250701809), VARSSINI A (250701811)” who carried out the project work under my supervision.

SIGNATURE	**SIGNATURE**
Internal Faculty Name	Internal Faculty Name
Course Incharge	Course Incharge
Object Oriented Programming System	Database Management System
Department of Computer Science and Engineering	Department of Computer Science and Engineering
INTERNAL EXAMINER	**EXTERNAL EXAMINER**

ABSTRACT

In today's world, peer learning and knowledge sharing systems are very important and essential for academic ecosystems. This management system is an application-based system to manage students and mark their skill portfolios. Every organisation whether government, private, or academic institution uses an information system to store and coordinate data of their members. This project is useful for providing an easy and responsive graphical user interface.

The system uses the powerful relational database management system, data retrieval, and data manipulation. This project provides more ease for managing skill exchange data than manually maintaining and searching across disconnected chat groups. Hence it saves a lot of valuable student time. So, we can say that the project is useful for saving valuable time and reducing huge paperwork.

The main objective of this integrated project is to add, update, delete, view, search, and match student skill details. The Skill Chain: Campus Peer-to-Peer Skill Exchange Platform is a comprehensive solution designed to streamline and automate various aspects of peer knowledge exchange within a university campus. This project aims to develop a user-friendly and efficient desktop platform that facilitates student authentication, maintenance of teaching and learning skill portfolios, automated two-way reciprocal matchmaking, exchange request negotiations, active partnership tracking, and instant notifications.

Skill Chain captures and stores detailed student information including student names, institutional emails, salted password hashes, teaching proficiencies, and learning aspirations. The system was developed using foundational technologies such as MySQL database and Java (JDBC) with Java Swing GUI.

TABLE OF CONTENTS

ABSTRACT 3

CHAPTER 1 - INTRODUCTION 6

PROBLEM STATEMENT 6

PROJECT OVERVIEW 6

PURPOSE 6

SCOPE 6

CHAPTER 2 - FUNCTIONALITIES PROVIDED 7

FUNCTIONALITIES PROVIDED BY SKILL CHAIN PLATFORM 7

CHAPTER 3 - SYSTEM FLOW DIAGRAM 7

DATAFLOW DIAGRAM 7

USE CASE DIAGRAM 8

ENTITY RELATIONSHIP DIAGRAM 8

CHAPTER 4 - MODULE DESCRIPTION 10

LOGIN & REGISTRATION MODULE 10

STUDENT DASHBOARD & ACTIVE PARTNERSHIPS MODULE 10

ADD SKILL MODULE 10

VIEW & SEARCH SKILLS MODULE 10

DELETE SKILL MODULE 10

RECURSIVE CATEGORY TREE TAXONOMY MODULE 11

CAMPUS STUDENT DIRECTORY MODULE 11

RECIPROCAL MATCHING ENGINE MODULE 11

LEARNING REQUESTS MODULE 11

NOTIFICATION CENTER MODULE 11

CHAPTER 5 - IMPLEMENTATION 11
MAIN LAUNCHER (Main.java) 11
JDBC CONNECTION CODE (DatabaseConnection.java) 12
SECURITY MODULE (PasswordUtil.java) 13
OOP MODEL: ABSTRACTION (Person.java) 14
OOP MODEL: INHERITANCE & POLYMORPHISM (User.java) 15
LOGIN & REGISTRATION PAGE 16
USER DATA ACCESS LAYER (UserDaoImpl.java) 20
DASHBOARD & ACTIVE PARTNERSHIPS 21
MY SKILLS PORTFOLIO (SkillService.java) 22
RECURSIVE CATEGORY TREE TAXONOMY (SkillTreeUtil.java) 25
CAMPUS STUDENT DIRECTORY 27
RECIPROCAL MATCHING ENGINE (MatchingService.java) 27
LEARNING REQUESTS (LearningRequestDaoImpl.java) 29
STUDENT NOTIFICATION CENTER 33
CHAPTER 6 - CONCLUSION 33
CHAPTER 7 - REFERENCES 34

(Right-click the table above in Word and select "Update Field" to refresh page numbers.)

CHAPTER 1 - INTRODUCTION

PROBLEM STATEMENT

On university campuses, students frequently seek to acquire specialized technical, mathematical, and design skills outside their formal syllabus. However, discovering compatible classmates who possess the desired skills and are simultaneously eager to learn what the first student can teach is extremely challenging. Informal inquiries across campus social media groups are unorganized, unindexed, lack confidentiality, and fail to calculate two-way reciprocal compatibility. An automated, centralized student skill exchange platform is required to manage student profiles, catalog skills, compute mutual matches, and track exchange agreements securely.

PROJECT OVERVIEW

The Skill Chain: Campus Peer-to-Peer Skill Exchange Platform is a comprehensive solution designed to streamline and automate various aspects of peer knowledge exchange within an educational institution. This project aims to develop a user-friendly and efficient desktop platform that facilitates student authentication, maintenance of personal skill inventories, automated two-way reciprocal matchmaking, proposal request negotiation, active partnership tracking, and real-time notification alerts.

Skill Chain captures and securely stores detailed student information including student names, verified university emails, cryptographically salted password hashes, and skill-type mappings. Through an automated relational Reciprocal Matching Engine, the platform pairs students when: Student A teaches what Student B wants to learn, and Student B teaches what Student A wants to learn.

PURPOSE

The system is aimed at total user-friendliness as well as efficient management of varied tasks:

- To centralize and organize student skill portfolios in a systematic manner.
- To automate the identification of reciprocal exchange partners.
- To provide a structured system for creating, reviewing, accepting, and declining skill exchange proposals.
- To track active learning partnerships showing contact emails and mutual study obligations.
- To safeguard sensitive student credentials through cryptographically salted SHA-256 password hashing.
- To allow for potential expansion and inclusion of additional categories.

SCOPE

The system is aimed at total user-friendliness as well as efficient management of different tasks:

- Capturing and storing essential student information including names, university emails, and password hashes.

• Providing a standardized academic catalog of skills categorized into programming, web development, data & AI, and digital design.
• Allowing students to tag skills as TEACH (skills offered) or LEARN (skills sought) with duplicate prevention.
• Enabling students to browse a campus-wide student directory with direct 'Send Request' action buttons.
• Executing automated dual-conditional matching queries using indexed MySQL joins.
• Managing exchange request lifecycles (PENDING, ACCEPTED, REJECTED) with notification indicators.
• Designing an intuitive aesthetic desktop interface using Java Swing.

CHAPTER 2 - FUNCTIONALITIES PROVIDED

FUNCTIONALITIES PROVIDED BY SKILL CHAIN PLATFORM

• Simplifying and streamlining peer skill exchanges.
• Provide timely notifications and reminders for incoming requests and accepted swaps.
• Easy to understand, access, and handle.
• Facilitate structured learning partnerships between university students.
• Automate reciprocal matchmaking calculations through relational multi-table self-joins.
• Centralize student skill portfolios with full CRUD capabilities (Create, Read, Update, Delete).
• Prevent duplicate skill registrations through database unique constraints.
• Safeguard student passwords with 16-byte random salt and SHA-256 cryptographic hashing.
• Support customizable skill categories and academic taxonomy trees.
• Provide real-time notification badges for pending and new exchange events.

CHAPTER 3 - SYSTEM FLOW DIAGRAM

System flow diagram helps in understanding the flow of information and processes within the system. The below displays such representations for the project on Skill Chain: Campus Peer-to-Peer Skill Exchange Platform.

DATAFLOW DIAGRAM

A Data Flow Diagram (DFD) is a visual way to show how data moves through a system or process. In Skill Chain, student inputs flow through authentication, skill management, matching engine, and request negotiation processes connected to normalized MySQL tables.

Level 0 (Context Diagram): Student → Skill Chain System → MySQL Database

Level 1 (Expanded): The student interacts with five core processes:

1. Authentication Process → reads/writes users table
2. Skill Portfolio Management → reads/writes user_skills and skills tables
3. Reciprocal Matching Engine → reads user_skills with 4-table self-join
4. Exchange Request Handler → reads/writes learning_requests table
5. Notification Service → reads learning_requests for status changes

USE CASE DIAGRAM

A use case diagram is a visual representation depicting the interactions between actors and a system, illustrating various ways the system can be utilized.

Actor: Student Member

Use Cases:
- Login to System – Allows an existing student to securely authenticate.
- Register New Account – Allows a new student to create a profile.
- View Dashboard & Metrics – Displays live metric counters for skills, matches, and requests.
- Add Skill – Allows a student to add a new TEACH or LEARN skill.
- View / Search My Skills – Enables a student to view and filter their skill inventory.
- Remove Skill – Permits a student to delete an existing skill.
- View Recursive Category Tree – Renders the academic taxonomy tree recursively.
- Browse Campus Directory – Displays all classmates with skill portfolios.
- View Reciprocal Matches – Automatically displays students with two-way compatibility.
- Send Exchange Proposal – Dispatches a formal exchange invitation.
- Accept / Reject Request – Allows receiving students to approve or decline proposals.
- View Notifications – Opens dynamic alerts regarding accepted swaps.
- Exit System – Allows a student to log out safely.

ENTITY RELATIONSHIP DIAGRAM

An entity-relationship diagram (ERD) is a visual representation illustrating the relationships between different entities within a database system.

Skill Chain has four core normalized relations with cascading referential integrity:

ENTITIES & ATTRIBUTES:
- users (user_id PK, name, email UNIQUE, password_hash, salt, created_at)
- skills (skill_id PK, skill_name, category, created_at)
- user_skills (user_skill_id PK, user_id FK, skill_id FK, skill_type, created_at)
- learning_requests (request_id PK, sender_id FK, receiver_id FK, offered_skill_id FK, requested_skill_id FK, status, created_at)

RELATIONSHIPS:

- users (1) ——< user_skills (many): One user has many skill associations
- skills (1) ——< user_skills (many): One skill can be held by many users
- users (1) ——< learning_requests (many): One user sends/receives many requests
- skills (1) ——< learning_requests (many): Each request references two skills

SQL SCHEMA:

```
CREATE TABLE users (
 user_id INT AUTO_INCREMENT PRIMARY KEY,
 name VARCHAR(100) NOT NULL,
 email VARCHAR(255) NOT NULL UNIQUE,
 password_hash VARCHAR(64) NOT NULL,
 salt VARCHAR(32) NOT NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE skills (
 skill_id INT AUTO_INCREMENT PRIMARY KEY,
 skill_name VARCHAR(100) NOT NULL,
 category VARCHAR(100) DEFAULT 'General',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE user_skills (
 user_skill_id INT AUTO_INCREMENT PRIMARY KEY,
 user_id INT NOT NULL,
 skill_id INT NOT NULL,
 skill_type ENUM('TEACH','LEARN') NOT NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
 FOREIGN KEY (skill_id) REFERENCES skills(skill_id) ON DELETE CASCADE,
 UNIQUE KEY unique_user_skill_type (user_id, skill_id, skill_type)
);

CREATE TABLE learning_requests (
 request_id INT AUTO_INCREMENT PRIMARY KEY,
 sender_id INT NOT NULL,
 receiver_id INT NOT NULL,
```

```
  offered_skill_id INT NOT NULL,
  requested_skill_id INT NOT NULL,
  status ENUM('PENDING','ACCEPTED','REJECTED') DEFAULT 'PENDING',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (sender_id) REFERENCES users(user_id) ON DELETE CASCADE,
  FOREIGN KEY (receiver_id) REFERENCES users(user_id) ON DELETE CASCADE,
  FOREIGN KEY (offered_skill_id) REFERENCES skills(skill_id) ON DELETE CASCADE,
  FOREIGN KEY (requested_skill_id) REFERENCES skills(skill_id) ON DELETE CASCADE
);
```

CHAPTER 4 - MODULE DESCRIPTION

The different types of modules present in this project are:

LOGIN & REGISTRATION MODULE

In this section, students can sign in or register new accounts with salted SHA-256 password protection. The module validates all inputs, checks for duplicate emails, generates a cryptographic salt, computes the SHA-256 hash, and stores credentials securely in the MySQL users table.

STUDENT DASHBOARD & ACTIVE PARTNERSHIPS MODULE

In this section, students can view dynamic KPI metric cards showing their total TEACH skills, LEARN skills, reciprocal matches count, and pending requests count. Active study partnerships (ACCEPTED requests) are displayed in a scrollable panel.

ADD SKILL MODULE

In this section, students can add skills to their TEACH or LEARN portfolio. The module prevents duplicate skill-type combinations using a MySQL UNIQUE constraint and throws DuplicateSkillException on violation.

VIEW & SEARCH SKILLS MODULE

In this section, students can inspect and filter their entire skill inventory. Overloaded search() methods allow keyword-only or keyword+category filtering using LIKE queries.

DELETE SKILL MODULE

In this section, students can delete a skill from their portfolio. An ownership check ensures only the skill owner can delete their own skill_id+user_id combination.

RECURSIVE CATEGORY TREE TAXONOMY MODULE

In this section, students can view the academic skill taxonomy rendered via recursive N-ary tree traversal. Three recursive methods are implemented: traverseTreeRecursive(), collectAllSkillsRecursive(), and findCategoryRecursive().

CAMPUS STUDENT DIRECTORY MODULE

In this section, students can browse all registered classmates with their teaching and learning skill summaries. Direct "Send Request" buttons enable immediate exchange proposal dispatch.

RECIPROCAL MATCHING ENGINE MODULE

In this section, an automated 4-table relational self-join query computes mutual learning compatibility. A match is found only when: (1) You teach what the partner wants to learn, AND (2) The partner teaches what you want to learn.

LEARNING REQUESTS MODULE

In this section, students can review incoming proposals (with Accept/Reject buttons) and track outgoing proposals. The status lifecycle is: PENDING → ACCEPTED or REJECTED.

NOTIFICATION CENTER MODULE

In this section, dynamic alerts keep students updated on their invitations and accepted swaps. Notifications are retrieved from learning_requests records with status changes.

CHAPTER 5 - IMPLEMENTATION

MAIN LAUNCHER (Main.java)

The application entry point configures the native system Look-and-Feel and launches the LoginRegisterFrame on the Swing Event Dispatch Thread (EDT).

```
package com.skillchain;

import com.skillchain.ui.LoginRegisterFrame;
import javax.swing.*;

/**
 * SKILL CHAIN - Application Entry Point (Main.java)
 */
public class Main {

    public static void main(String[] args) {
        // Set native system look-and-feel for clean UI integration
        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        } catch (Exception ignored) {
            // Fall back to default Swing look and feel
        }

        // Launch Desktop Swing Application on Event Dispatch Thread (EDT)
        SwingUtilities.invokeLater(() -> {
```

```
            LoginRegisterFrame loginFrame = new LoginRegisterFrame();
            loginFrame.setVisible(true);
        });
    }
}
```

JDBC CONNECTION CODE (DatabaseConnection.java)

Reads MySQL credentials from config.properties. Never hardcodes passwords. Implements a robust two-stage configuration loading strategy (external file, then classpath fallback). Provides testConnection() and saveConfiguration() helper methods.

```
package com.skillchain.util;

import java.io.*;
import java.sql.*;
import java.util.Properties;

/**
 * DATABASE CONNECTION MANAGER (JDBC)
 * Reads database credentials securely from config.properties.
 */
public final class DatabaseConnection {

    private static final String CONFIG_FILE_NAME = "config.properties";
    private static String dbUrl, dbUser, dbPassword;
    private static boolean initialized = false;

    private DatabaseConnection() {}

    private static synchronized void initialize() {
        if (initialized) return;
        Properties properties = new Properties();
        File externalConfigFile = new File(CONFIG_FILE_NAME);
        try {
            if (externalConfigFile.exists()) {
                try (InputStream fis = new FileInputStream(externalConfigFile)) {
                    properties.load(fis);
                }
            } else {
                try (InputStream cis = DatabaseConnection.class
                        .getClassLoader().getResourceAsStream(CONFIG_FILE_NAME)) {
                    if (cis != null) {
                        properties.load(cis);
                    } else {
                        throw new DatabaseException("config.properties not found!");
                    }
                }
            }
            dbUrl      = properties.getProperty("db.url");
            dbUser     = properties.getProperty("db.user");
            dbPassword = properties.getProperty("db.password", "");

            if (dbUrl == null || dbUser == null) {
                throw new DatabaseException("db.url and db.user must be set.");
            }
            Class.forName("com.mysql.cj.jdbc.Driver");
            initialized = true;
        } catch (IOException e) {
            throw new DatabaseException("Failed to read config: " + e.getMessage(), e);
        } catch (ClassNotFoundException e) {
            throw new DatabaseException("MySQL Driver not found on classpath.", e);
        }
```

```
    }

    /** Obtains a new JDBC Connection. Use with try-with-resources. */
    public static Connection getConnection() throws SQLException {
        if (!initialized) initialize();
        return DriverManager.getConnection(dbUrl, dbUser, dbPassword);
    }

    /** Diagnostic connectivity check. */
    public static boolean testConnection() {
        try (Connection conn = getConnection()) {
            return conn != null && !conn.isClosed();
        } catch (Exception e) {
            return false;
        }
    }

    /** Saves user-supplied credentials to config.properties at runtime. */
    public static synchronized void saveConfiguration(
            String url, String user, String password) {
        Properties p = new Properties();
        p.setProperty("db.url",      url.trim());
        p.setProperty("db.user",     user.trim());
        p.setProperty("db.password", password == null ? "" : password);
        try (OutputStream out = new FileOutputStream(CONFIG_FILE_NAME)) {
            p.store(out, "Skill Chain local database configuration");
            initialized = false;
            initialize();
        } catch (IOException e) {
            throw new DatabaseException("Cannot save config: " + e.getMessage(), e);
        }
    }

    /** Re-reads config.properties if changed at runtime. */
    public static synchronized void reloadConfig() {
        initialized = false;
        initialize();
    }
}
```

SECURITY MODULE (PasswordUtil.java)

Implements SHA-256 password hashing with a 16-byte cryptographic salt. Uses SecureRandom, MessageDigest, and constant-time comparison (MessageDigest.isEqual) to guard against timing attacks.

```
package com.skillchain.util;

import java.nio.charset.StandardCharsets;
import java.security.*;
import java.util.HexFormat;

/**
 * SECURITY: SHA-256 PASSWORD HASHING WITH CRYPTOGRAPHIC SALT
 * - SecureRandom generates a unique 16-byte cryptographically secure salt.
 * - MessageDigest computes the SHA-256 digest of salt + password.
 * - Constant-time comparison prevents timing-based side-channel attacks.
 */
public final class PasswordUtil {

    private static final int SALT_BYTE_LENGTH = 16;
    private static final SecureRandom SECURE_RANDOM = new SecureRandom();
```

```
    private PasswordUtil() {}

    /** Generates a cryptographically strong 32-character hex salt. */
    public static String generateSalt() {
        byte[] salt = new byte[SALT_BYTE_LENGTH];
        SECURE_RANDOM.nextBytes(salt);
        return HexFormat.of().formatHex(salt);
    }

    /**
     * Computes SHA-256 hash of password concatenated with salt.
     * @param plainPassword  Plaintext password
     * @param saltHex        Hex-encoded salt
     * @return 64-character lowercase hexadecimal SHA-256 hash
     */
    public static String hashPassword(String plainPassword, String saltHex) {
        if (plainPassword == null || saltHex == null)
            throw new IllegalArgumentException("Password and salt must not be null.");
        try {
            byte[] saltBytes = HexFormat.of().parseHex(saltHex);
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            digest.update(saltBytes);
            byte[] hashBytes = digest.digest(
                    plainPassword.getBytes(StandardCharsets.UTF_8));
            return HexFormat.of().formatHex(hashBytes);
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException("SHA-256 unavailable on JVM", e);
        }
    }

    /**
     * Verifies a candidate password against stored salt and hash.
     * Uses constant-time MessageDigest.isEqual to prevent timing attacks.
     */
    public static boolean verifyPassword(
            String rawPassword, String saltHex, String expectedHashHex) {
        if (rawPassword == null || saltHex == null || expectedHashHex == null)
            return false;
        String calculatedHash = hashPassword(rawPassword, saltHex);
        byte[] calc = calculatedHash.getBytes(StandardCharsets.UTF_8);
        byte[] exp  = expectedHashHex.getBytes(StandardCharsets.UTF_8);
        return MessageDigest.isEqual(calc, exp); // constant-time
    }
}
```

OOP MODEL: ABSTRACTION (Person.java)

Person is an abstract base class demonstrating the Abstraction OOP concept. It defines shared identity attributes and enforces abstract methods getRole() and getSummary() on all concrete subclasses.

```
package com.skillchain.model;

import java.time.LocalDateTime;

/**
 * OOP CONCEPT: ABSTRACTION
 * Person is an abstract class serving as the base model.
 * It prevents instantiation of incomplete identities while
 * providing an extensible foundation for authenticated users.
 */
public abstract class Person {
```

```
    private int id;
    private String name;
    private String email;
    private LocalDateTime createdAt;

    public Person() {}

    public Person(int id, String name, String email, LocalDateTime createdAt) {
        setId(id); setName(name); setEmail(email);
        this.createdAt = createdAt;
    }

    /** Abstract: Every concrete subclass must declare its system role. */
    public abstract String getRole();

    /** Abstract: Every concrete subclass must provide an identity summary. */
    public abstract String getSummary();

    // ENCAPSULATION: Getters and Setters with Business Validation
    public int getId() { return id; }
    public void setId(int id) {
        if (id < 0) throw new IllegalArgumentException("ID cannot be negative.");
        this.id = id;
    }

    public String getName() { return name; }
    public void setName(String name) {
        if (name == null || name.trim().length() < 2)
            throw new IllegalArgumentException("Name must be >= 2 characters.");
        this.name = name.trim();
    }

    public String getEmail() { return email; }
    public void setEmail(String email) {
        if (email == null || !email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$"))
            throw new IllegalArgumentException("Invalid email format.");
        this.email = email.trim().toLowerCase();
    }

    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }

    @Override
    public String toString() {
        return String.format("%s (ID: %d, Email: %s)", name, id, email);
    }
}
```

OOP MODEL: INHERITANCE & POLYMORPHISM (User.java)

User extends Person demonstrating Inheritance. It overrides getRole() and getSummary() (Runtime Polymorphism / Method Overriding) and adds security credential fields (passwordHash, salt).

```
package com.skillchain.model;

import java.time.LocalDateTime;

/**
 * OOP CONCEPTS: INHERITANCE & METHOD OVERRIDING (POLYMORPHISM)
 * User extends Person, inheriting id, name, email, createdAt while
 * adding security credentials: passwordHash and salt.
 */
```

```
public class User extends Person {

    private String passwordHash;
    private String salt;

    public User() { super(); }

    public User(int id, String name, String email,
                String passwordHash, String salt, LocalDateTime createdAt) {
        super(id, name, email, createdAt);
        setPasswordHash(passwordHash);
        setSalt(salt);
    }

    /** POLYMORPHISM: Overrides abstract getRole() from Person */
    @Override
    public String getRole() { return "Student Member (Peer Exchange)"; }

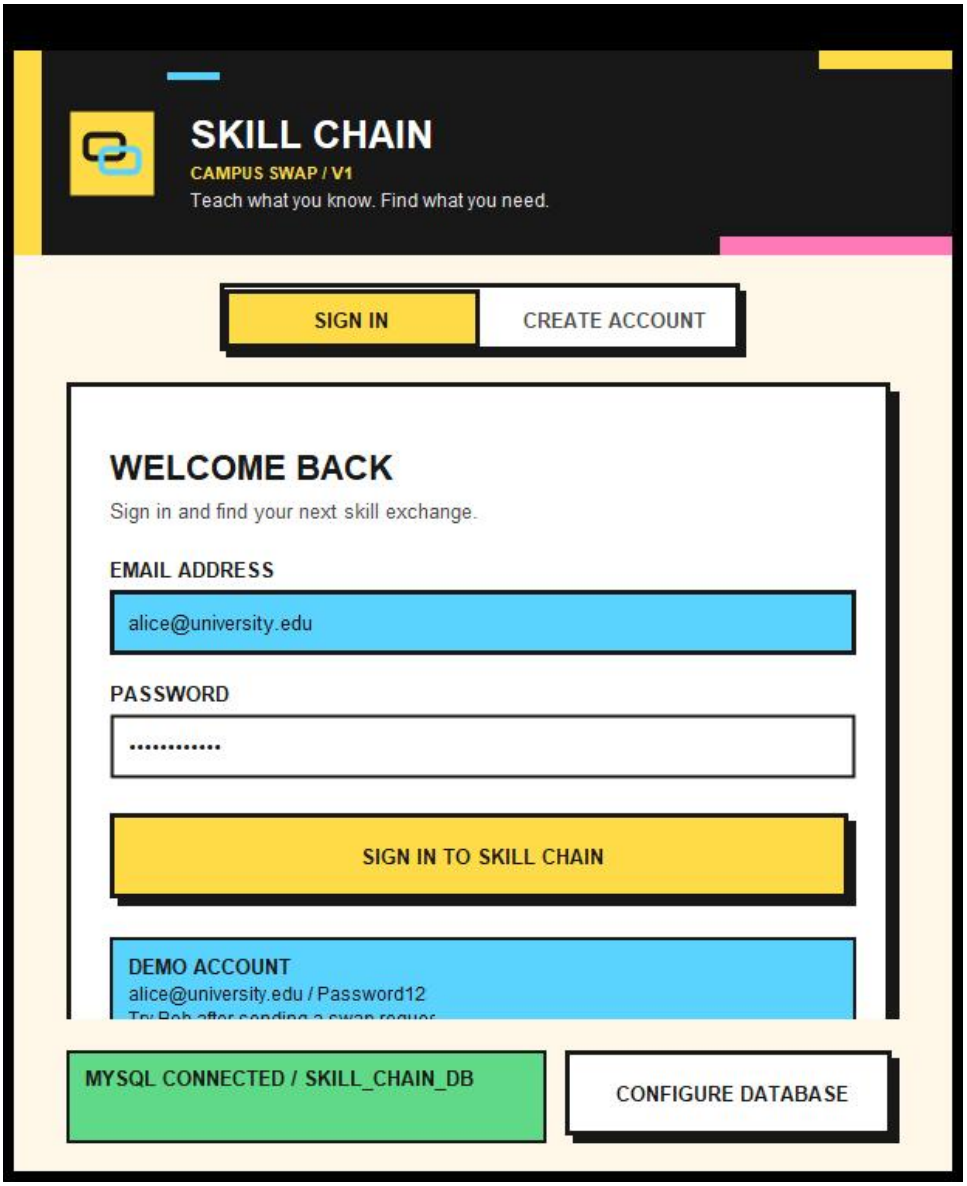
    /** POLYMORPHISM: Overrides abstract getSummary() from Person */
    @Override
    public String getSummary() {
        return String.format("Student Member: %s (%s)", getName(), getEmail());
    }

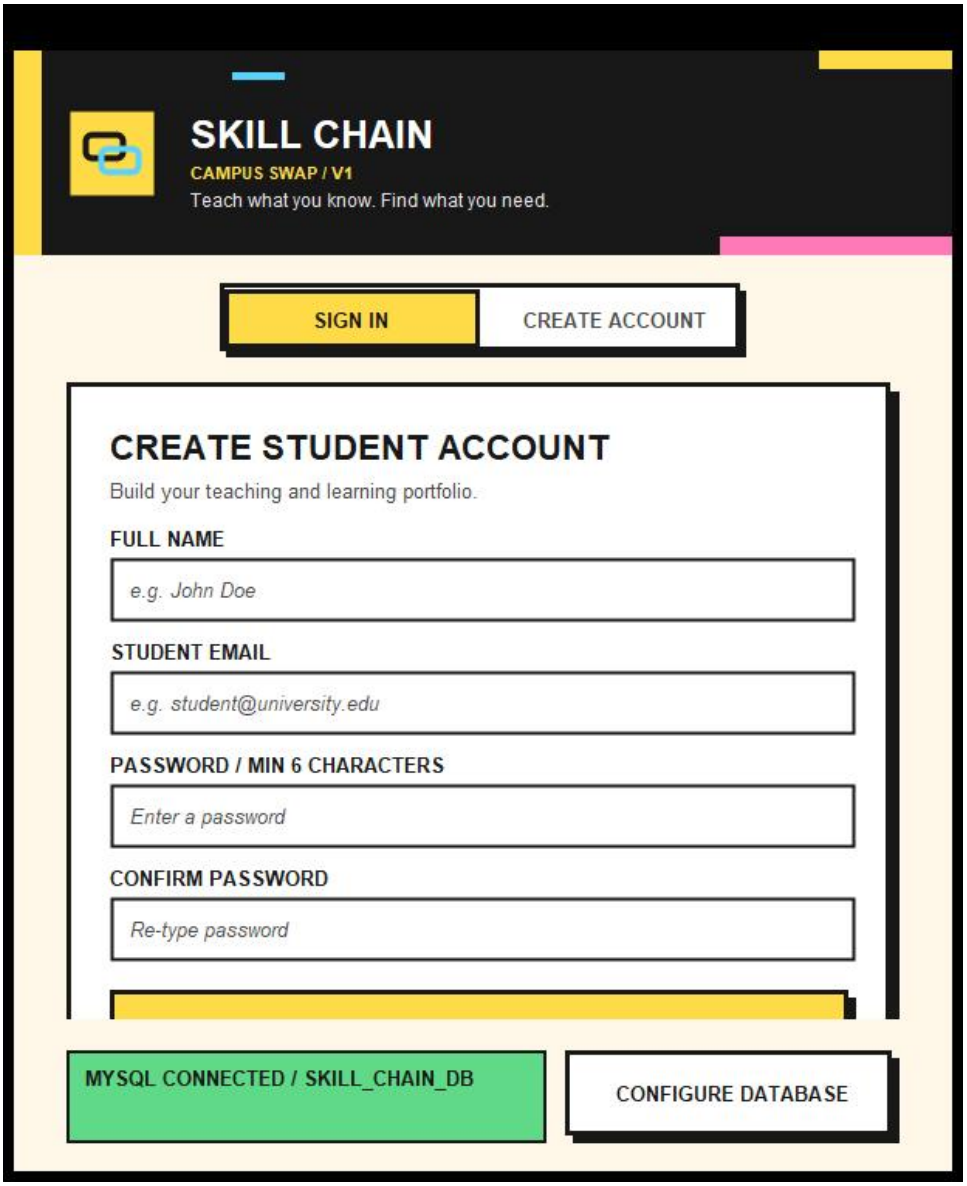
    // ENCAPSULATION with defensive validation
    public String getPasswordHash() { return passwordHash; }
    public void setPasswordHash(String passwordHash) {
        if (passwordHash == null || passwordHash.trim().isEmpty())
            throw new IllegalArgumentException("Password hash cannot be empty.");
        this.passwordHash = passwordHash.trim();
    }

    public String getSalt() { return salt; }
    public void setSalt(String salt) {
        if (salt == null || salt.trim().isEmpty())
            throw new IllegalArgumentException("Salt cannot be empty.");
        this.salt = salt.trim();
    }

    @Override
    public String toString() {
        return String.format("User{id=%d, name='%s', email='%s', role='%s'}",
                getId(), getName(), getEmail(), getRole());
    }
}
```

LOGIN & REGISTRATION PAGE


SKILL CHAIN
CAMPUS SWAP / V1
Teach what you know. Find what you need.
SIGN IN
CREATE ACCOUNT
WELCOME BACK
Sign in and find your next skill exchange.
EMAIL ADDRESS
alice@university.edu
PASSWORD
SIGN IN TO SKILL CHAIN
DEMO ACCOUNT
alice@university.edu / Password12
MYSQL CONNECTED / SKILL_CHAIN_DB
CONFIGURE DATABASE



The UserService class handles all authentication business logic. The login() method retrieves the stored salt and hash from the database and uses PasswordUtil.verifyPassword() for constant-time comparison.

```
package com.skillchain.service;

import com.skillchain.dao.UserDao;
import com.skillchain.dao.UserDaoImpl;
import com.skillchain.model.User;
import com.skillchain.util.*;

import java.sql.SQLException;
import java.util.Optional;

/**
 * SERVICE LAYER: AUTHENTICATION & USER MANAGEMENT
 */
```

```
public class UserService {

    private final UserDao userDao;

    public UserService() { this(new UserDaoImpl()); }
    public UserService(UserDao userDao) { this.userDao = userDao; }

    /**
     * Registers a new student account with salted SHA-256 hashing.
     */
    public User register(String name, String email,
                         String password, String confirmPassword)
            throws DuplicateEmailException, SQLException {

        if (name == null || name.trim().length() < 2)
            throw new IllegalArgumentException("Name must be >= 2 characters.");
        if (email == null || !email.matches(
                "^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$"))
            throw new IllegalArgumentException("Please enter a valid email address.");
        if (password == null || password.length() < 6)
            throw new IllegalArgumentException("Password must be >= 6 characters.");
        if (!password.equals(confirmPassword))
            throw new IllegalArgumentException("Passwords do not match.");

        if (userDao.existsByEmail(email))
            throw new DuplicateEmailException("Email '" + email + "' already registered.");

        String salt         = PasswordUtil.generateSalt();
        String passwordHash = PasswordUtil.hashPassword(password, salt);

        User newUser = new User();
        newUser.setName(name.trim());
        newUser.setEmail(email.trim().toLowerCase());
        newUser.setPasswordHash(passwordHash);
        newUser.setSalt(salt);
        return userDao.save(newUser);
    }

    /**
     * Authenticates a student using email and plaintext password.
     */
    public User login(String email, String password)
            throws InvalidLoginException, SQLException {

        if (email == null || email.isBlank() || password == null || password.isEmpty())
            throw new InvalidLoginException("Please provide both email and password.");

        Optional<User> userOpt = userDao.findByEmail(email.trim().toLowerCase());
        if (userOpt.isEmpty())
            throw new InvalidLoginException("Invalid email or password.");

        User user = userOpt.get();
        boolean ok = PasswordUtil.verifyPassword(
                password, user.getSalt(), user.getPasswordHash());
        if (!ok)
            throw new InvalidLoginException("Invalid email or password.");

        return user;
    }
}
```

JDBC INSERT CODE (Registration):

```
INSERT INTO users (name, email, password_hash, salt) VALUES (?, ?, ?, ?);
```

JDBC SELECT CODE (Login):

```
SELECT user_id, name, email, password_hash, salt, created_at FROM users WHERE LOWER(email) = ?;
```

USER DATA ACCESS LAYER (UserDaoImpl.java)

UserDaoImpl implements the UserDao interface using JDBC PreparedStatement and try-with-resources. Demonstrates proper resource management, duplicate key detection (MySQL error 1062), and GROUP_CONCAT for peer skill summaries.

```
package com.skillchain.dao;

import com.skillchain.model.*;
import com.skillchain.util.*;

import java.sql.*;
import java.util.*;

/**
 * JDBC implementation of UserDao.
 * Uses PreparedStatements and try-with-resources.
 */
public class UserDaoImpl implements UserDao {

    @Override
    public User save(User user) throws DuplicateEmailException, SQLException {
        if (existsByEmail(user.getEmail()))
            throw new DuplicateEmailException("Email already exists: " + user.getEmail());

        final String sql =
            "INSERT INTO users (name, email, password_hash, salt) VALUES (?, ?, ?, ?)";

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(
                     sql, Statement.RETURN_GENERATED_KEYS)) {

            stmt.setString(1, user.getName());
            stmt.setString(2, user.getEmail().toLowerCase());
            stmt.setString(3, user.getPasswordHash());
            stmt.setString(4, user.getSalt());
            stmt.executeUpdate();

            try (ResultSet gk = stmt.getGeneratedKeys()) {
                if (gk.next()) user.setId(gk.getInt(1));
            }
            return user;
        } catch (SQLException e) {
            if (e.getErrorCode() == 1062) // MySQL ER_DUP_ENTRY
                throw new DuplicateEmailException("Email already exists.", e);
            throw e;
        }
    }

    @Override
    public Optional<User> findByEmail(String email) throws SQLException {
        final String sql =
            "SELECT user_id,name,email,password_hash,salt,created_at "
          + "FROM users WHERE LOWER(email)=?";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, email.toLowerCase());
            try (ResultSet rs = stmt.executeQuery()) {
                while (rs.next()) return Optional.of(mapRowToUser(rs));
```

```
            }
        }
        return Optional.empty();
    }

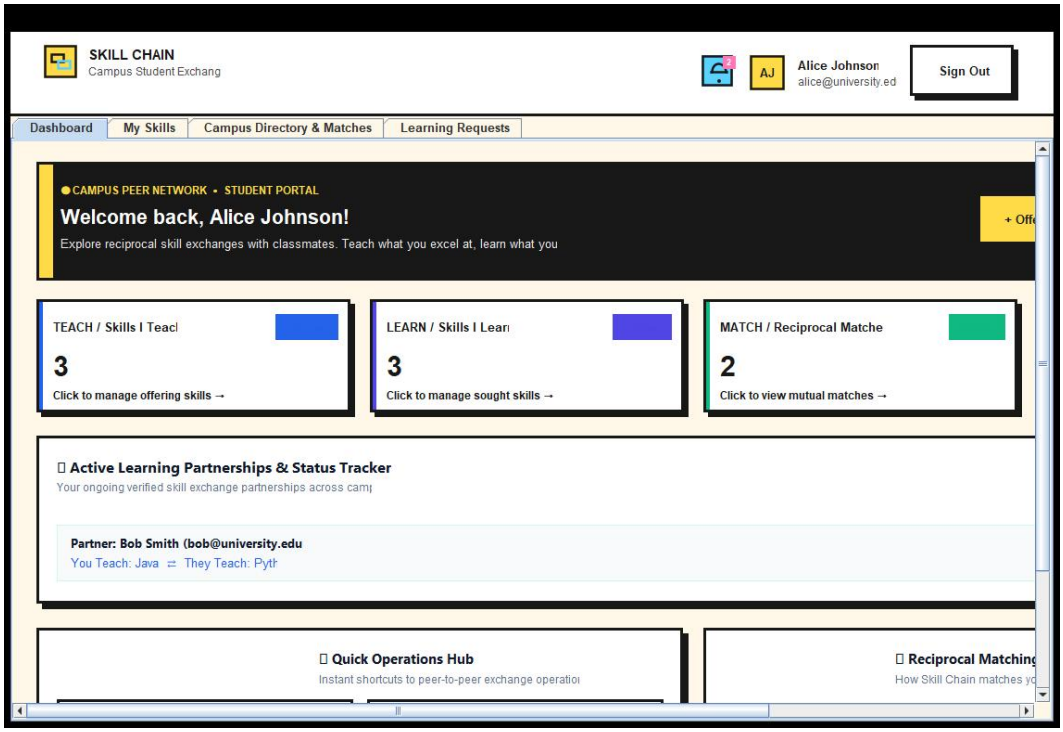
    @Override
    public boolean existsByEmail(String email) throws SQLException {
        final String sql = "SELECT 1 FROM users WHERE LOWER(email)=? LIMIT 1";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, email.toLowerCase());
            try (ResultSet rs = stmt.executeQuery()) { return rs.next(); }
        }
    }

    @Override
    public List<PeerStudent> findAllPeersWithSkills(int excludeUserId)
            throws SQLException {
        final String sql =
            "SELECT u.user_id, u.name, u.email, "
          + "COALESCE(GROUP_CONCAT(DISTINCT CASE WHEN us.skill_type='TEACH' "
          + "         THEN s.skill_name END SEPARATOR ', '),'None listed yet') AS teach_skills,"
          + "COALESCE(GROUP_CONCAT(DISTINCT CASE WHEN us.skill_type='LEARN' "
          + "         THEN s.skill_name END SEPARATOR ', '),'None listed yet') AS learn_skills "
          + "FROM users u "
          + "LEFT JOIN user_skills us ON u.user_id=us.user_id "
          + "LEFT JOIN skills s ON us.skill_id=s.skill_id "
          + "WHERE u.user_id!=? "
          + "GROUP BY u.user_id,u.name,u.email ORDER BY u.name ASC";

        List<PeerStudent> peers = new ArrayList<>();
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setInt(1, excludeUserId);
            try (ResultSet rs = stmt.executeQuery()) {
                while (rs.next())
                    peers.add(new PeerStudent(rs.getInt("user_id"),
                            rs.getString("name"), rs.getString("email"),
                            rs.getString("teach_skills"),rs.getString("learn_skills")));
            }
        }
        return peers;
    }

    private User mapRowToUser(ResultSet rs) throws SQLException {
        Timestamp ts = rs.getTimestamp("created_at");
        return new User(rs.getInt("user_id"), rs.getString("name"),
                rs.getString("email"), rs.getString("password_hash"),
                rs.getString("salt"),
                ts != null ? ts.toLocalDateTime() : null);
    }
}
```

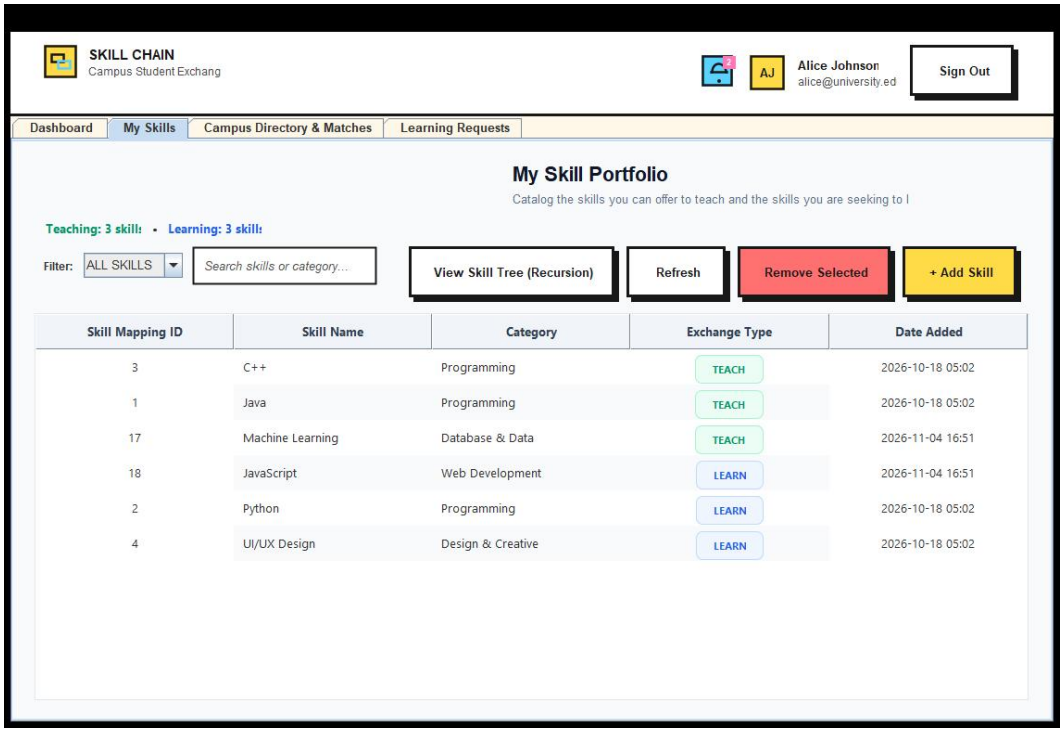
DASHBOARD & ACTIVE PARTNERSHIPS



JDBC QUERY CODE (Active Partnerships):

```
SELECT lr.request_id,
       CASE WHEN lr.sender_id=? THEN u_recv.name ELSE u_send.name END AS partner_name,
       CASE WHEN lr.sender_id=? THEN u_recv.email ELSE u_send.email END AS partner_email,
       CASE WHEN lr.sender_id=? THEN s_off.skill_name ELSE s_req.skill_name END AS you_teach,
       CASE WHEN lr.sender_id=? THEN s_req.skill_name ELSE s_off.skill_name END AS
partner_teaches,
       lr.status, lr.created_at
FROM learning_requests lr
JOIN users u_send ON lr.sender_id = u_send.user_id
JOIN users u_recv ON lr.receiver_id = u_recv.user_id
JOIN skills s_off ON lr.offered_skill_id = s_off.skill_id
JOIN skills s_req ON lr.requested_skill_id = s_req.skill_id
WHERE (lr.sender_id=? OR lr.receiver_id=?) AND lr.status='ACCEPTED'
ORDER BY lr.created_at DESC;
```

MY SKILLS PORTFOLIO (SkillService.java)



SkillService demonstrates METHOD OVERLOADING (Compile-Time Polymorphism). Two versions of searchSkills() and getUserSkills() accept different parameters, dispatching to the appropriate DAO method.

```
package com.skillchain.service;

import com.skillchain.dao.*;
import com.skillchain.model.*;
import com.skillchain.util.*;

import java.sql.SQLException;
import java.util.*;

/**
 * SERVICE LAYER: SKILL CATALOG & PORTFOLIO MANAGEMENT
 * Demonstrates METHOD OVERLOADING (Compile-Time Polymorphism)
 */
public class SkillService {

    private final SkillDao skillDao;

    public SkillService() { this(new SkillDaoImpl()); }
    public SkillService(SkillDao skillDao) { this.skillDao = skillDao; }

    public List<Skill> getAllSkills() throws SQLException {
        return skillDao.findAll();
    }

    // ===== METHOD OVERLOADING (Polymorphism) =====

    /** Overload 1: Keyword search across ALL categories. */
    public List<Skill> searchSkills(String keyword) throws SQLException {
        return skillDao.search(keyword);
    }
```

```
    /** Overload 2: Keyword search restricted to a specific category. */
    public List<Skill> searchSkills(String keyword, String category)
            throws SQLException {
        return skillDao.search(keyword, category);
    }

    // ===== USER SKILLS MANAGEMENT =====

    /** Adds an existing catalog skill to the user's TEACH or LEARN list. */
    public void addUserSkill(int userId, int skillId, SkillType skillType)
            throws DuplicateSkillException, SQLException {
        if (userId <= 0 || skillId <= 0 || skillType == null)
            throw new IllegalArgumentException("Invalid user, skill, or type.");
        skillDao.addUserSkill(userId, skillId, skillType);
    }

    /** Adds a custom skill: creates in catalog if not found, then attaches. */
    public void addCustomUserSkill(int userId, String skillName,
            String category, SkillType skillType)
            throws DuplicateSkillException, SQLException {
        if (skillName == null || skillName.trim().length() < 2)
            throw new IllegalArgumentException("Skill name must be >= 2 chars.");
        String name = skillName.trim();
        String cat  = (category == null || category.isBlank()) ? "General" : category.trim();
        Optional<Skill> existing = skillDao.findByName(name);
        Skill skill = existing.isPresent()
                ? existing.get()
                : skillDao.createSkill(new Skill(name, cat));
        skillDao.addUserSkill(userId, skill.getSkillId(), skillType);
    }

    /** Removes a skill from the user's portfolio (ownership enforced in DAO). */
    public void removeUserSkill(int userSkillId, int userId) throws SQLException {
        skillDao.removeUserSkill(userSkillId, userId);
    }

    /** Overload 1: All skills for user (both TEACH and LEARN). */
    public List<UserSkill> getUserSkills(int userId) throws SQLException {
        return skillDao.getUserSkills(userId);
    }

    /** Overload 2: Only skills of a specific type. */
    public List<UserSkill> getUserSkills(int userId, SkillType skillType)
            throws SQLException {
        return skillDao.getUserSkills(userId, skillType);
    }

    /** Builds and returns the recursive category taxonomy string. */
    public String getFormattedSkillTaxonomyTree() {
        SkillCategoryNode root = SkillTreeUtil.createDefaultSkillTree();
        StringBuilder buffer = new StringBuilder();
        buffer.append("=== ACADEMIC SKILL TAXONOMY ===\n\n");
        SkillTreeUtil.traverseTreeRecursive(root, 0, buffer);
        return buffer.toString();
    }
}
```

JDBC INSERT (Add Skill):

```
INSERT INTO user_skills (user_id, skill_id, skill_type) VALUES (?, ?, ?);
```

JDBC SELECT (View Skills):

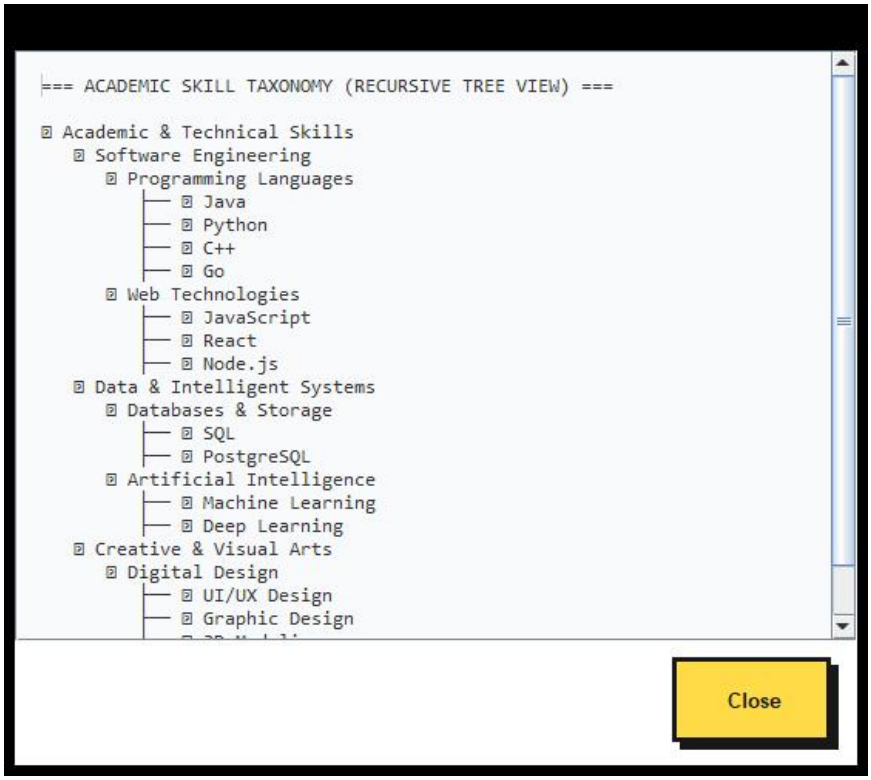
```
SELECT us.user_skill_id, us.user_id, us.skill_id, s.skill_name, s.category,
       us.skill_type, us.created_at
```

```
FROM user_skills us
JOIN skills s ON us.skill_id = s.skill_id
WHERE us.user_id = ?
ORDER BY us.skill_type ASC, s.skill_name ASC;
```

JDBC DELETE (Remove Skill):

```
DELETE FROM user_skills WHERE user_skill_id = ? AND user_id = ?;
```

RECURSIVE CATEGORY TREE TAXONOMY (SkillTreeUtil.java)



SkillTreeUtil demonstrates three recursive algorithms over a hierarchical N-ary skill category tree. Recursion is the most mathematically elegant way to traverse trees of arbitrary depth.

```
package com.skillchain.util;

import com.skillchain.model.SkillCategoryNode;
import java.util.*;

/**
 * OOP CONCEPT: RECURSION
 * Three recursive methods traverse an N-ary skill category tree.
 */
public final class SkillTreeUtil {

    private SkillTreeUtil() {}

    /** Builds the default academic skill category tree. */
    public static SkillCategoryNode createDefaultSkillTree() {
        SkillCategoryNode root = new SkillCategoryNode("Academic & Technical Skills");
```

```
    SkillCategoryNode software = new SkillCategoryNode("Software Engineering");
    SkillCategoryNode prog = new SkillCategoryNode("Programming Languages");
    prog.addSkill("Java"); prog.addSkill("Python");
    prog.addSkill("C++"); prog.addSkill("Go");
    SkillCategoryNode web = new SkillCategoryNode("Web Technologies");
    web.addSkill("JavaScript"); web.addSkill("React"); web.addSkill("Node.js");
    software.addChild(prog); software.addChild(web);

    SkillCategoryNode data = new SkillCategoryNode("Data & Intelligent Systems");
    SkillCategoryNode db = new SkillCategoryNode("Databases & Storage");
    db.addSkill("SQL"); db.addSkill("PostgreSQL");
    SkillCategoryNode ai = new SkillCategoryNode("Artificial Intelligence");
    ai.addSkill("Machine Learning"); ai.addSkill("Deep Learning");
    data.addChild(db); data.addChild(ai);

    SkillCategoryNode creative = new SkillCategoryNode("Creative & Visual Arts");
    SkillCategoryNode design = new SkillCategoryNode("Digital Design");
    design.addSkill("UI/UX Design"); design.addSkill("Graphic Design");
    design.addSkill("3D Modeling");
    creative.addChild(design);

    root.addChild(software); root.addChild(data); root.addChild(creative);
    return root;
}

/**
 * RECURSIVE METHOD 1: Tree Visualization
 * Base Case: node == null -> return.
 * Recursive Step: traverseTreeRecursive(child, depth+1, buffer)
 */
public static void traverseTreeRecursive(
        SkillCategoryNode node, int depth, StringBuilder buffer) {
    if (node == null) return; // Base Case

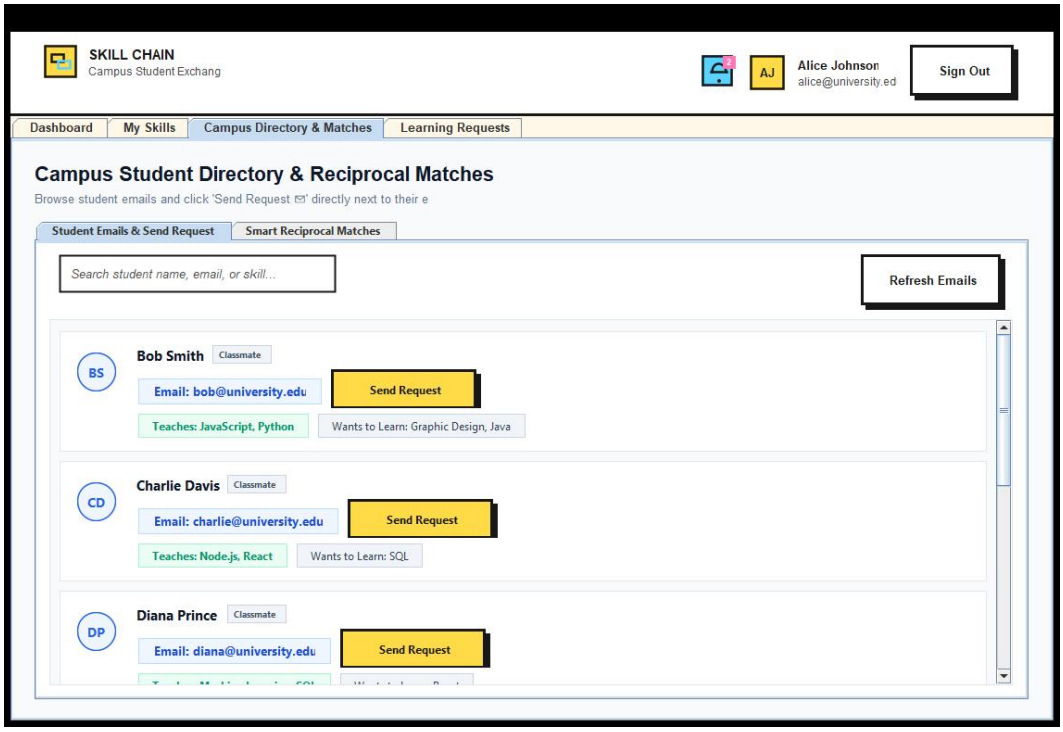
    String indent = "   ".repeat(depth);
    buffer.append(indent).append("[+] ").append(node.getName()).append("\n");
    for (String skill : node.getSkills())
        buffer.append(indent).append("   |-- ").append(skill).append("\n");
    for (SkillCategoryNode child : node.getChildren())
        traverseTreeRecursive(child, depth + 1, buffer); // Recursive Step
}

/**
 * RECURSIVE METHOD 2: Skill Collection
 * Recursively accumulates all leaf skill names from any subtree.
 */
public static List<String> collectAllSkillsRecursive(SkillCategoryNode node) {
    List<String> skills = new ArrayList<>();
    if (node == null) return skills;
    skills.addAll(node.getSkills());
    for (SkillCategoryNode child : node.getChildren())
        skills.addAll(collectAllSkillsRecursive(child));
    return skills;
}

/**
 * RECURSIVE METHOD 3: Subtree Search
 * Recursively searches for a category node matching targetName.
 */
public static SkillCategoryNode findCategoryRecursive(
        SkillCategoryNode current, String targetName) {
    if (current == null || targetName == null) return null;
    if (current.getName().equalsIgnoreCase(targetName.trim())) return current;
    for (SkillCategoryNode child : current.getChildren()) {
        SkillCategoryNode found = findCategoryRecursive(child, targetName);
```

```
            if (found != null) return found;
        }
        return null; // Not found in this branch
    }
}
```

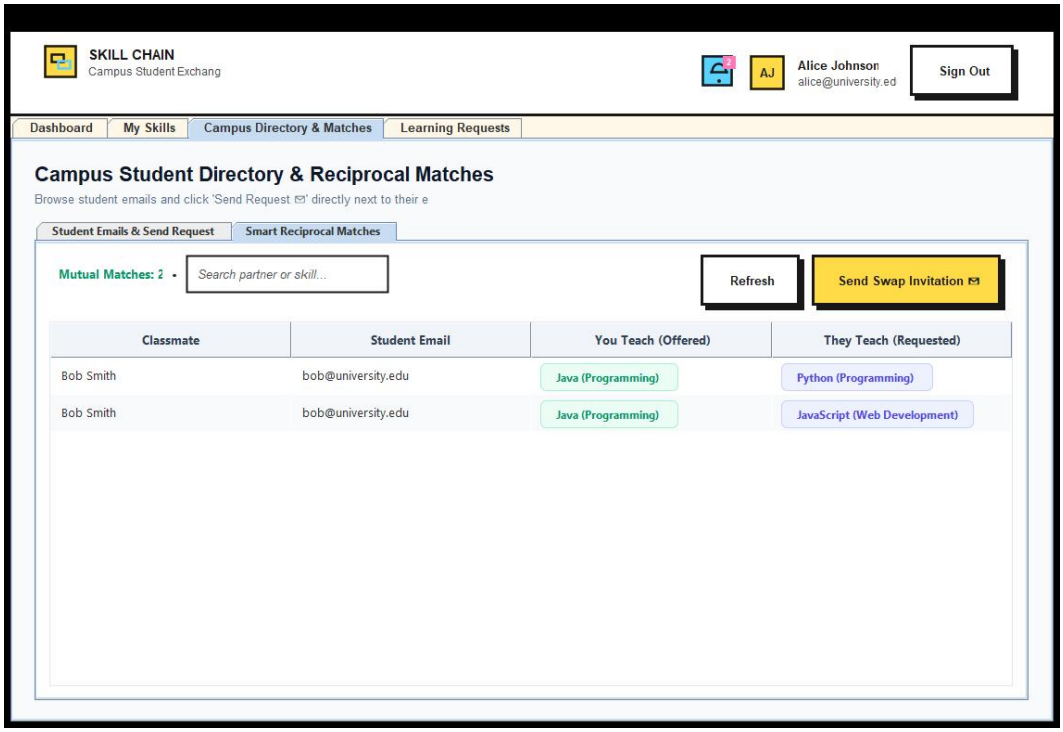
CAMPUS STUDENT DIRECTORY



JDBC QUERY CODE (Campus Directory with GROUP_CONCAT):

```
SELECT u.user_id, u.name, u.email,
       COALESCE(GROUP_CONCAT(DISTINCT CASE WHEN us.skill_type = 'TEACH'
           THEN s.skill_name END SEPARATOR ', '), 'None listed yet') AS teach_skills,
       COALESCE(GROUP_CONCAT(DISTINCT CASE WHEN us.skill_type = 'LEARN'
           THEN s.skill_name END SEPARATOR ', '), 'None listed yet') AS learn_skills
FROM users u
LEFT JOIN user_skills us ON u.user_id = us.user_id
LEFT JOIN skills s ON us.skill_id = s.skill_id
WHERE u.user_id != ?
GROUP BY u.user_id, u.name, u.email
ORDER BY u.name ASC;
```

RECIPROCAL MATCHING ENGINE (MatchingService.java)



MatchingService executes a 4-table self-join SQL query to identify mutual exchange opportunities. A match requires BOTH conditions: (1) You teach what the partner needs to learn, AND (2) The partner teaches what you need to learn.

```
package com.skillchain.service;

import com.skillchain.model.ReciprocalMatch;
import com.skillchain.util.DatabaseConnection;

import java.sql.*;
import java.util.*;

/**
 * SERVICE LAYER: SMART RECIPROCAL MATCHING ENGINE
 * Discovers mutual exchange opportunities via a 4-table self-join.
 */
public class MatchingService {

    private static final String RECIPROCAL_MATCH_SQL =
        "SELECT u_partner.user_id AS partner_id, "
      + "       u_partner.name AS partner_name, "
      + "       u_partner.email AS partner_email, "
      + "       s_offer.skill_id AS user_offered_id, "
      + "       s_offer.skill_name AS user_offered_name, "
      + "       s_offer.category AS user_offered_category, "
      + "       s_partner.skill_id AS partner_offered_id, "
      + "       s_partner.skill_name AS partner_offered_name, "
      + "       s_partner.category AS partner_offered_category "
      + "FROM user_skills my_teach "
      + "JOIN user_skills partner_learn "
      + "    ON my_teach.skill_id = partner_learn.skill_id "
      + "   AND my_teach.skill_type = 'TEACH' "
      + "   AND partner_learn.skill_type = 'LEARN' "
      + "   AND partner_learn.user_id != my_teach.user_id "
      + "JOIN user_skills partner_teach "
```

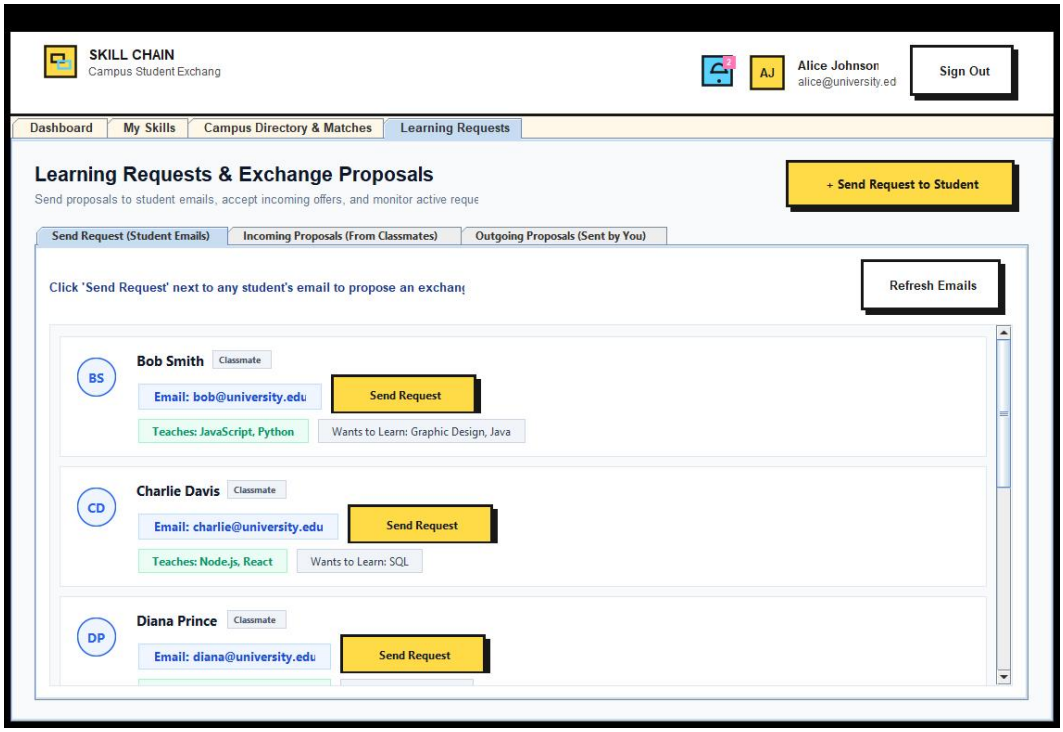
```
      + "    ON partner_teach.user_id = partner_learn.user_id "
      + "   AND partner_teach.skill_type = 'TEACH' "
      + "JOIN user_skills my_learn "
      + "    ON my_learn.skill_id = partner_teach.skill_id "
      + "   AND my_learn.user_id = my_teach.user_id "
      + "   AND my_learn.skill_type = 'LEARN' "
      + "JOIN users u_partner ON u_partner.user_id = partner_learn.user_id "
      + "JOIN skills s_offer  ON s_offer.skill_id  = my_teach.skill_id "
      + "JOIN skills s_partner ON s_partner.skill_id = partner_teach.skill_id "
      + "WHERE my_teach.user_id = ? "
      + "ORDER BY u_partner.name ASC";

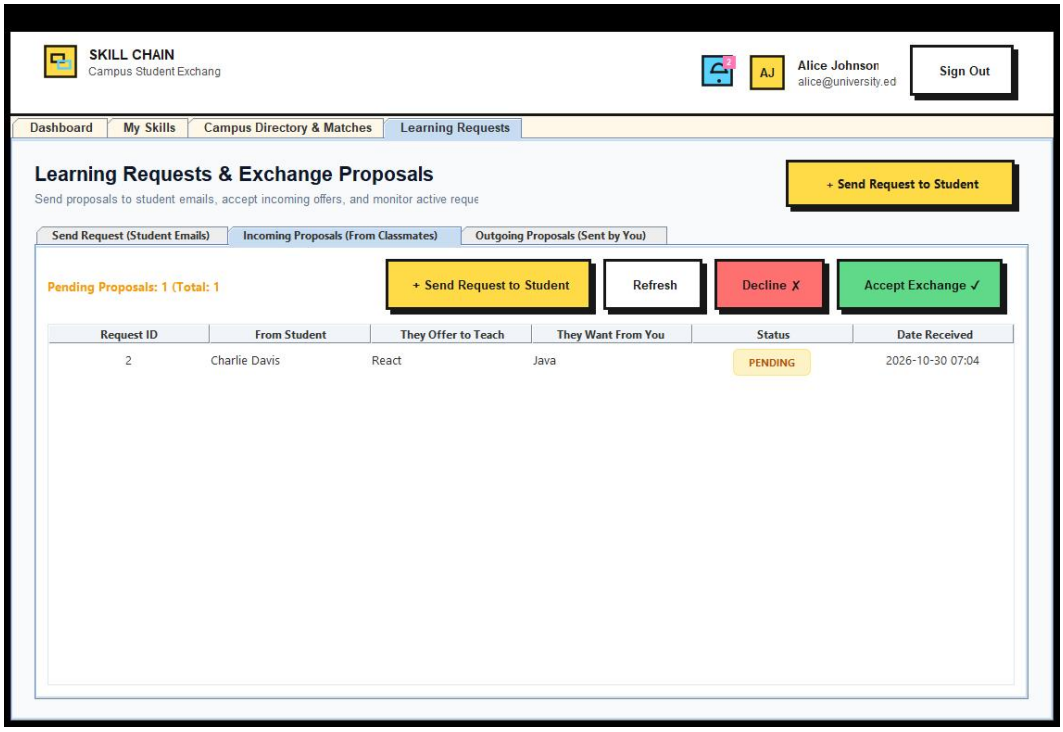
    /** Finds all reciprocal matches for the given student. */
    public List<ReciprocalMatch> findReciprocalMatches(int currentUserId)
            throws SQLException {
        List<ReciprocalMatch> matches = new ArrayList<>();
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(RECIPROCAL_MATCH_SQL)) {

            stmt.setInt(1, currentUserId);
            try (ResultSet rs = stmt.executeQuery()) {
                while (rs.next()) {
                    matches.add(new ReciprocalMatch(
                            rs.getInt("partner_id"),
                            rs.getString("partner_name"),
                            rs.getString("partner_email"),
                            rs.getInt("user_offered_id"),
                            rs.getString("user_offered_name"),
                            rs.getString("user_offered_category"),
                            rs.getInt("partner_offered_id"),
                            rs.getString("partner_offered_name"),
                            rs.getString("partner_offered_category")
                    ));
                }
            }
        }
        return matches;
    }

    /** Quick KPI count for dashboard metric card. */
    public int getReciprocalMatchCount(int currentUserId) throws SQLException {
        return findReciprocalMatches(currentUserId).size();
    }
}
```

LEARNING REQUESTS (LearningRequestDaoImpl.java)





LearningRequestDaoImpl manages the full lifecycle of exchange proposals (PENDING -> ACCEPTED / REJECTED) using JDBC PreparedStatements with multi-table joins.

```
package com.skillchain.dao;
```

```
import com.skillchain.model.LearningRequest;
import com.skillchain.util.DatabaseConnection;

import java.sql.*;
import java.time.LocalDateTime;
import java.util.*;

/**
 * JDBC DATA ACCESS: LEARNING REQUESTS
 * Implements LearningRequestDao using PreparedStatement and try-with-resources.
 */
public class LearningRequestDaoImpl implements LearningRequestDao {

    private static final String BASE_SQL =
        "SELECT lr.request_id, lr.sender_id, u_send.name AS sender_name, "
      + "       lr.receiver_id, u_recv.name AS receiver_name, "
      + "       lr.offered_skill_id, s_off.skill_name AS offered_skill_name, "
      + "       lr.requested_skill_id, s_req.skill_name AS requested_skill_name, "
      + "       lr.status, lr.created_at "
      + "FROM learning_requests lr "
      + "JOIN users u_send ON lr.sender_id   = u_send.user_id "
      + "JOIN users u_recv ON lr.receiver_id = u_recv.user_id "
      + "JOIN skills s_off ON lr.offered_skill_id   = s_off.skill_id "
      + "JOIN skills s_req ON lr.requested_skill_id = s_req.skill_id ";

    @Override
    public LearningRequest save(LearningRequest request) throws SQLException {
        final String sql =
            "INSERT INTO learning_requests "
          + "(sender_id,receiver_id,offered_skill_id,requested_skill_id,status) "
          + "VALUES (?,?,?,?,?)";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(
                     sql, Statement.RETURN_GENERATED_KEYS)) {
            stmt.setInt(1, request.getSenderId());
            stmt.setInt(2, request.getReceiverId());
            stmt.setInt(3, request.getOfferedSkillId());
            stmt.setInt(4, request.getRequestedSkillId());
            stmt.setString(5, request.getStatus() != null ? request.getStatus() : "PENDING");
            stmt.executeUpdate();
            try (ResultSet gk = stmt.getGeneratedKeys()) {
                if (gk.next()) request.setRequestId(gk.getInt(1));
            }
            if (request.getCreatedAt() == null) request.setCreatedAt(LocalDateTime.now());
            return request;
        }
    }

    @Override
    public List<LearningRequest> findIncomingByUserId(int receiverId)
            throws SQLException {
        return executeQuery(BASE_SQL + "WHERE lr.receiver_id=? ORDER BY lr.created_at DESC",
                receiverId);
    }

    @Override
    public List<LearningRequest> findOutgoingByUserId(int senderId)
            throws SQLException {
        return executeQuery(BASE_SQL + "WHERE lr.sender_id=? ORDER BY lr.created_at DESC",
                senderId);
    }

    @Override
    public boolean updateStatus(int requestId, String newStatus) throws SQLException {
        final String sql = "UPDATE learning_requests SET status=? WHERE request_id=?";
```

```
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, newStatus);
            stmt.setInt(2, requestId);
            return stmt.executeUpdate() > 0;
        }
    }

    @Override
    public int countPendingIncoming(int receiverId) throws SQLException {
        final String sql =
            "SELECT COUNT(*) FROM learning_requests WHERE receiver_id=? AND status='PENDING'";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setInt(1, receiverId);
            try (ResultSet rs = stmt.executeQuery()) {
                if (rs.next()) return rs.getInt(1);
            }
        }
        return 0;
    }

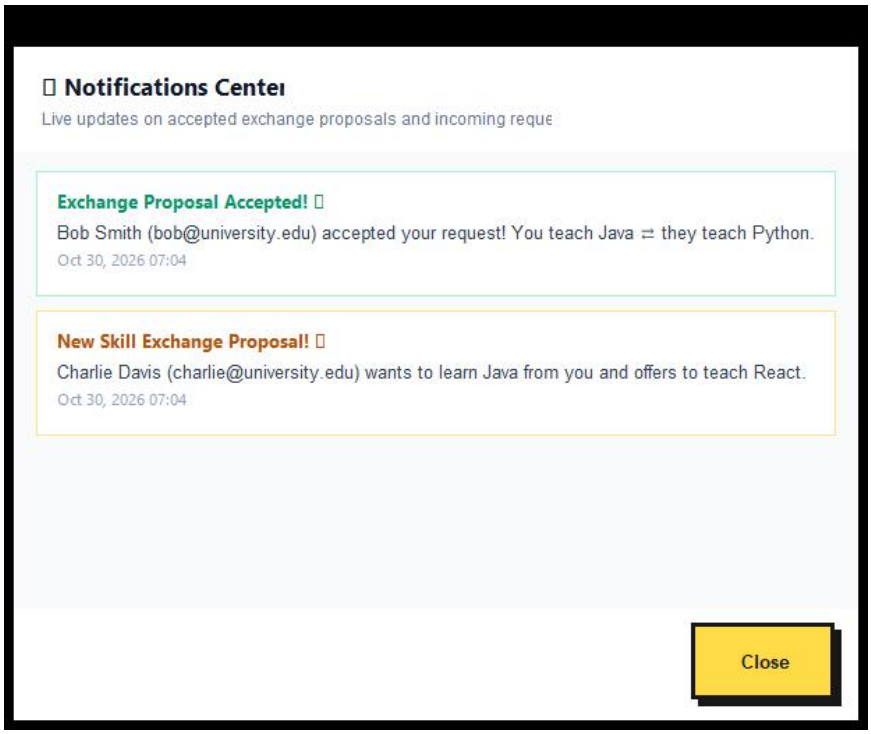
    @Override
    public boolean existsActiveRequest(int senderId, int receiverId,
            int offeredSkillId, int requestedSkillId) throws SQLException {
        final String sql =
            "SELECT 1 FROM learning_requests "
          + "WHERE sender_id=? AND receiver_id=? "
          + "  AND offered_skill_id=? AND requested_skill_id=? "
          + "  AND status IN ('PENDING','ACCEPTED') LIMIT 1";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setInt(1, senderId); stmt.setInt(2, receiverId);
            stmt.setInt(3, offeredSkillId); stmt.setInt(4, requestedSkillId);
            try (ResultSet rs = stmt.executeQuery()) { return rs.next(); }
        }
    }

    private List<LearningRequest> executeQuery(String sql, int userId)
            throws SQLException {
        List<LearningRequest> list = new ArrayList<>();
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setInt(1, userId);
            try (ResultSet rs = stmt.executeQuery()) {
                while (rs.next()) {
                    Timestamp ts = rs.getTimestamp("created_at");
                    list.add(new LearningRequest(
                            rs.getInt("request_id"), rs.getInt("sender_id"),
                            rs.getString("sender_name"), rs.getInt("receiver_id"),
                            rs.getString("receiver_name"), rs.getInt("offered_skill_id"),
                            rs.getString("offered_skill_name"),
                            rs.getInt("requested_skill_id"),
                            rs.getString("requested_skill_name"),
                            rs.getString("status"),
                            ts != null ? ts.toLocalDateTime() : LocalDateTime.now()));
                }
            }
        }
        return list;
    }
}
```

JDBC STATUS UPDATE:

```
UPDATE learning_requests SET status = ? WHERE request_id = ?;
```

STUDENT NOTIFICATION CENTER



The Notification Center displays real-time alerts for all accepted swaps and pending incoming proposals by querying the learning_requests table filtered by the current user's receiver_id.

CHAPTER 6 - CONCLUSION

In conclusion, our student skill exchange database management system implemented in Java using JDBC successfully addresses the core functionalities of adding new skills, removing records, updating statuses, and efficiently retrieving data when needed. The utilization of JDBC allowed seamless interaction between the Java application and the MySQL database engine, ensuring robust data management and transaction security.

The project emphasized the importance of adopting core Object-Oriented Programming (OOPS) principles:

• Abstraction: separating interface contracts from database implementations (Person abstract class).

• Inheritance: modeling domain users through base class reuse (User extends Person).

• Encapsulation: defensive input validation and getter/setter boundaries across all models.

• Polymorphism: compile-time method overloading (searchSkills, getUserSkills) and runtime method overriding (getRole, getSummary).

• Data Structures & Recursion: traversing nested academic category trees using three depth-first recursive algorithms.
• Application Security: cryptographically salted SHA-256 password hashing using SecureRandom.

Additionally, the project's reliance on established database management principles ensured a solid foundation for data integrity, 3rd Normal Form (3NF) relational design, referential cascading constraints, and multi-table join query optimization.

Through this project, we gained valuable insights into Java application development, database connectivity, and the significance of adhering to best practices in software design. The system provides a user-friendly interface for managing student skill portfolios, contributing to a more efficient and collaborative peer learning process on campus.

CHAPTER 7 - REFERENCES

1. JAVA – The Complete Reference by ORACLE Press / Herbert Schildt.
2. Database System Concepts by Abraham Silberschatz, Henry F. Korth, and S. Sudarshan (McGraw-Hill).
3. MySQL 8.0 Reference Manual – Oracle Corporation.
4. YouTube Playlist on Java Swing and JDBC Architecture: https://www.youtube.com/playlist?list=PLbKub4Jss9oUbTmeS9li_DoskYCiuvx1R